

Lab0: GitLab

睦罗涛 25303090064

TODO1:

(1) 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：有过较为基础的多人协同开发经历，主要是在规定了全局变量的基础上，各自分工写不同的函数，最后在 main 函数中进行调用，得到完整的程序。

(2) Git 为什么要设计“暂存-提交”两个步骤？

答：设计“暂存-提交”两个步骤，可以实现选择性提交，可以只将 A 部分放进本次 commit 而 B 部分留到下一次提交；同时，暂存区相当于对本次提交进行预览，便于检查即将提交的部分是否存在失误和不足；而且暂存区的存在允许我们在编写一个部分的中途切换到另一个分支去处理可能更为紧急的问题，而原来的部分就可以先存放到暂存区，等其他分支处理完后再切换回来继续编写。

(3) git branch 和 git branch -a 的区别是什么？

答：git branch 只查看电脑本地创建的分支；git branch -a 可以查看本地分支（不带 remotes/）和远程追踪分支（前缀 remotes/origin/xxx 只标记远程仓库上存在的分支）。

TODO2:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("旦子真是肥嘟嘟的\n");
7  }
```

TODO3:

(1) 第一篇：Git Commit Message 规范

这篇文章主要陈述如何写出规范、清晰的提交说明。其核心内容包括：格式结构（由 Header、Body、Footer 三部分组成），其中 Header 必填；七种 type 标识：feat（新功能）、fix（修 bug）、docs（文档）、style（格式）、refactor（重构）、test（测试）、chore（构建/工具变动）；规则细节：subject 用动词原形、首字母小写、不加句号；不兼容变动要用 BREAKING CHANGE 说明；撤销提交用 revert：开头以及相应配套工具。

(2) 第二篇：Gitflow 分支控制

这篇文章讲的是多人协作下的分支管理规范。其核心内容包括：分支体系：master（线上代码，打 tag）、develop（开发主分支）是永久分支；feature（新功能）、release（提测/修 bug）、hotfix（线上紧急修复）是临时分支，用完即删；核心原则是“从哪里来，最后回到哪里去”——feature 从 develop 创建、合并回 develop；hotfix 从 master 创建、合并回 master 和 develop；合并统一用 --no-ff 保留分支历史；实操方式既可以用原生 Git 命令操作，也可以用 git flow 扩展库简化流程；版本号用 0.0.0 三段式，在 master 上打 tag。

(3) 为什么要学习使用 Git？

首先，在没有版本管理时，代码改坏了只能靠记忆或备份还原，试错成本极高。而 Git 的每一次 commit 都是一份快照，随时可以回溯到任意历史版本，极大的降低了重构、实验、探索的风险成本。其次，Git 契合现代软件开发的多人协作模式。Git 的分支与合并机制，让多人并行开发互不干扰，再通过合并汇总成果。最后，Git 使工作流规范化，进而带来团队效率，分支各司其职、commit 规范统一，使得出问题能快速定位是哪次提交引入的、发布时自动生成变更日志、新成员入职后按流程就能上手，降低沟通成本。

TODO4:

创建并切换到 feature 分支

```
hilbert@LAPTOP-4PUAMS9C:~/25303090064$ git branch feature
hilbert@LAPTOP-4PUAMS9C:~/25303090064$ git switch feature
Switched to branch 'feature'
hilbert@LAPTOP-4PUAMS9C:~/25303090064$
```

修改 main 并提交

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("hello feature\n");
7 }
```

切换到 main 分支

```
hilbert@LAPTOP-4PUAMS9C:~/25303090064$ git switch main
Switched to branch 'main'
```

修改 main 并提交

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("hello main \n");
7 }
```

合并时遇到冲突

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     <<<<<<< HEAD (当前更改)
7     printf("hello main \n");
8     =====
9     printf("hello feature\n");
10    >>>>>> feature (传入的更改)
11 }
```

解决合并冲突（变为 feature main）

```
传入 7f97707 - refs/heads/feature
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("hello feature\n");
7 }

当前 14ef872 - refs/heads/main
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("hello main \n");
7 }

结果 main.c
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("且子真是肥嘟嘟的\n");
7 }
```

```
hilbert@LAPTOP-4PUAMS9C:~/25303090064$ git add main.c
hilbert@LAPTOP-4PUAMS9C:~/25303090064$ git commit -m"解决合并冲突"
[main 6e24449] 解决合并冲突
```

出现冲突的原因：同一位置在不同分支中被多次修改